

Tag Quiet Timer And Quiet/Unquiet Design

Scope

This document describes a safer and cleaner implementation for:

- Setting a tag quiet timer from UI input.
- Performing periodic quiet operations on selected tags.
- Performing unquiet operations to restore tag visibility.

The design uses your existing classes:

- InventoryFragment
- InventoryViewModel
- Zebra API calls through `impinjExtensions.setTagQuiet(...)`

Current API Mapping

The current implementation already uses the correct Zebra entry points:

- Quiet: `STATE_AWARE_ACTION.STATE_AWARE_ACTION`
- Unquiet: `STATE_AWARE_ACTION.STATE_AWARE_ACTION_DSRT_SL`
- Optional second unquiet reset step with `TARGET_INVENTORIED_STATE_S3` and `STATE_AWARE_ACTION_INV_A`

Recommended mask defaults (same as current code):

- `ENUM_TAGQUIET_MASK.S3B`

Design Goals

1. Keep RFID operations off the UI thread.
2. Parse timer values once and validate bounds.
3. Make quiet and unquiet explicit operations.
4. Avoid conflicting inventory transitions while stop is in progress.
5. Keep ViewModel as the owner of RFID behavior; Fragment only orchestrates UI events.

Proposed Data Model

```
public final class QuietConfig {
    public final int quietIntervalMs;
    public final int inventoryStopLimitSec;

    public QuietConfig(int quietIntervalMs, int inventoryStopLimitSec) {
        this.quietIntervalMs = quietIntervalMs;
        this.inventoryStopLimitSec = inventoryStopLimitSec;
    }

    public static QuietConfig fromUi(String quietTimerSec, String stopLimitSec) {
        int quietSec = 2; // default 2s
        int stopSec = 0; // 0 means disabled

        try {
            if (quietTimerSec != null && !quietTimerSec.trim().isEmpty()) {
                quietSec = Integer.parseInt(quietTimerSec.trim());
            }
        } catch (NumberFormatException ignored) {}

        try {
            if (stopLimitSec != null && !stopLimitSec.trim().isEmpty()) {
                stopSec = Integer.parseInt(stopLimitSec.trim());
            }
        } catch (NumberFormatException ignored) {}

        // Clamp bounds to prevent accidental extreme values.
        quietSec = Math.max(1, Math.min(quietSec, 600));
    }
}
```

```

        stopSec = Math.max(0, Math.min(stopSec, 86400));

        return new QuietConfig(quietSec * 1000, stopSec);
    }
}

```

ViewModel: Quiet Operation

Use a dedicated method that receives selected EPCs and applies prefilters + quiet in one place.

```

public boolean applyQuietToSelectedTags(ArrayList<InventoryItem> selectedItems)
    throws InvalidUsageException, OperationFailureException {
    if (mConnectedRfidReader == null || !mConnectedRfidReader.isConnected()) {
        return false;
    }
    if (selectedItems == null || selectedItems.isEmpty()) {
        return false;
    }

    // 1) Clear old prefilters.
    mConnectedRfidReader.Actions.PreFilters.deleteAll();

    // 2) Add EPC prefilters for selected tags.
    PreFilters.PreFilter[] preFiltersArray = new PreFilters.PreFilter[selectedItems.size()];
    for (int i = 0; i < selectedItems.size(); i++) {
        InventoryItem item = selectedItems.get(i);
        PreFilters preFilters = new PreFilters();
        PreFilters.PreFilter preFilter = preFilters.new PreFilter();
        preFilter.setAntennaID((short) 1);
        preFilter.setBitOffset(32);
        preFilter.setTagPatternBitCount(96);
        preFilter.setTagPattern(item.getEPC());
        preFilter.setMemoryBank(MEMORY_BANK.MEMORY_BANK_EPC);
        preFilter.setFilterAction(FILTER_ACTION.FILTER_ACTION_STATE_AWARE);
        preFilter.StateAwareAction.setTarget(TARGET.TARGET_INVENTORIED_STATE_S3);
        preFilter.StateAwareAction.setStateAwareAction(STATE_AWARE_ACTION.STATE_AWARE_ACTION_INV_B);
        preFiltersArray[i] = preFilter;
    }
    mConnectedRfidReader.Actions.PreFilters.add(preFiltersArray, null);

    // 3) Execute TagQuiet.
    ENUM_TAGQUIET_MASK[] tagMask = { ENUM_TAGQUIET_MASK.S3B };
    impinjExtensions.setTagQuiet(
        tagMask,
        TARGET.TARGET_SL,
        STATE_AWARE_ACTION.STATE_AWARE_ACTION_ASRT_SL,
        (short) 1
    );

    // 4) Keep singulation aligned with quiet behavior.
    setSingulation(
        SESSION.SESSION_S2,
        INVENTORY_STATE.INVENTORY_STATE_AB_FLIP,
        SL_FLAG.SL_FLAG_DEASSERTED
    );
}

```

```

    return true;
}

```

ViewModel: Unquiet Operation

Unquiet should be directly callable and idempotent.

```

public boolean applyUnquiet()
    throws InvalidUsageException, OperationFailureException {
    if (mConnectedRfidReader == null || !mConnectedRfidReader.isConnected()) {
        return false;
    }

    ENUM_TAGQUIET_MASK[] tagMask = { ENUM_TAGQUIET_MASK.S3B };

    // 1) Deassert SL quiet state.
    impinjExtensions.setTagQuiet(
        tagMask,
        TARGET.TARGET_SL,
        STATE_AWARE_ACTION.STATE_AWARE_ACTION_DSRT_SL,
        (short) 1
    );

    // 2) Reset inventoried state for S3 path.
    impinjExtensions.setTagQuiet(
        tagMask,
        TARGET.TARGET_INVENTORIED_STATE_S3,
        STATE_AWARE_ACTION.STATE_AWARE_ACTION_INV_A,
        (short) 1
    );

    // Optional: clear prefilters if needed by UX semantics.
    mConnectedRfidReader.Actions.PreFilters.deleteAll();
    return true;
}

```

Fragment: Timer Setup And Loop

Parse once when starting the cycle, then reuse the computed interval.

```

private QuietConfig activeQuietConfig;
private final Handler quietHandler = new Handler(Looper.getMainLooper());

private final Runnable quietCycleRunnable = new Runnable() {
    @Override
    public void run() {
        if (binding == null || inventoryViewModel.inventoryBeingStopped) {
            quietHandler.removeCallbacks(this);
            return;
        }
        if (!binding.selectAll.isChecked()) {
            quietHandler.removeCallbacks(this);
            return;
        }
    }
}

```

```

        executor.execute(() -> {
            try {
                ArrayList<InventoryItem> selected = inventoryViewModel.getCheckedItems().getValue();
                inventoryViewModel.applyQuietToSelectedTags(selected);
            } catch (Exception e) {
                Log.e(TAG, "quietCycleRunnable failed", e);
            }
        });

        quietHandler.postDelayed(this, activeQuietConfig.quietIntervalMs);
    }
};

private void startQuietCycleFromUi() {
    activeQuietConfig = QuietConfig.fromUi(
        binding.quietTimer.getText().toString(),
        binding.inventoryStopTimer.getText().toString()
    );

    quietHandler.removeCallbacks(quietCycleRunnable);
    quietCycleRunnable.run(); // immediate first cycle
}

private void stopQuietCycle() {
    quietHandler.removeCallbacks(quietCycleRunnable);
}

```

Fragment: Button Wiring Example

This keeps button actions explicit and predictable.

```

private void setupButtons() {
    binding.buttonQuiet.setOnClickListener(v -> {
        if (mConnectedRfidReader == null || !mConnectedRfidReader.isConnected()) {
            Snackbar.make(v, "Reader Not Connected", Snackbar.LENGTH_SHORT).show();
            return;
        }
        inventoryViewModel.inventoryBeingStopped = false;
        startQuietCycleFromUi();
    });

    binding.checkboxUnquiet.setOnCheckedChangeListener((button, checked) -> {
        if (!checked) return;
        executor.execute(() -> {
            try {
                inventoryViewModel.applyUnquiet();
            } catch (Exception e) {
                Log.e(TAG, "applyUnquiet failed", e);
            }
        });
    });
}

```